

```
In [1]: # importing the iris dataset
from sklearn.datasets import load_iris
import pandas as pd
dataset = load_iris() #creating a object
cols = dataset.feature_names #names of the input features
df = pd.DataFrame(dataset.data, columns=cols)
#dataset.data - value of the input features
```

```
In [2]: df.head()
```

```
Out[2]:
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

```
In [3]: df.tail()
```

```
Out[3]:
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

```
In [4]: df['target'] = dataset.target
# will have the output values
```

```
In [5]: df.head()
```

```
Out[5]:
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```
In [6]: df.tail()
```

```
Out[6]:
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
145	6.7	3.0	5.2	2.3	2
146	6.3	2.5	5.0	1.9	2
147	6.5	3.0	5.2	2.0	2
148	6.2	3.4	5.4	2.3	2
149	5.9	3.0	5.1	1.8	2

```
In [7]: from sklearn.preprocessing import StandardScaler
# all data should on same scale (Important step)
x=df.iloc[:, :-1]
#all except last column
```

```
In [8]: print(x)
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
...	...	...	...	...
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

[150 rows x 4 columns]

```
In [10]: y=df.iloc[:,-1]
# only last column
print(y)
```

0	0
1	0
2	0
3	0
4	0
...	...
145	2
146	2
147	2
148	2
149	2

Name: target, Length: 150, dtype: int32

```
In [11]: x=StandardScaler().fit_transform(x) # Scaling X
x=pd.DataFrame(x)
print(x)
```

	0	1	2	3
0	-0.900681	1.019004	-1.340227	-1.315444
1	-1.143017	-0.131979	-1.340227	-1.315444
2	-1.385353	0.328414	-1.397064	-1.315444
3	-1.506521	0.098217	-1.283389	-1.315444
4	-1.021849	1.249201	-1.340227	-1.315444
...	...	...	...	...
145	1.038005	-0.131979	0.819596	1.448832
146	0.553333	-1.282963	0.705921	0.922303
147	0.795669	-0.131979	0.819596	1.053935
148	0.432165	0.788808	0.933271	1.448832
149	0.068662	-0.131979	0.762758	0.790671

[150 rows x 4 columns]

```
In [12]: #Applying PCA
from sklearn.decomposition import PCA
pca = PCA() #object
#just on x- pca comes under unsupervised learning
x_pca = pca.fit_transform(x) #applying pca on x
x_pca = pd.DataFrame(x_pca) #converting into a dataframe
```

```
In [13]: #shows the components along with the loading score
pca.components_
```

```
Out[13]: array([[ 0.52106591, -0.26934744,  0.5804131 ,  0.56485654],
 [ 0.37741762,  0.92329566,  0.02449161,  0.06694199],
 [-0.71956635,  0.24438178,  0.14212637,  0.63427274],
 [-0.26128628,  0.12350962,  0.80144925, -0.52359713]])
```

```
In [14]: #variance covered by each component
print('Explained Variance', pca.explained_variance_) # Variation (found by ssd/n-1)
print('Proportion of Explained Variance',pca.explained_variance_ratio_) #proportional form (eg.v1/v1+v2+v3)

Explained Variance [2.93808505 0.9201649  0.14774182 0.02085386]
Proportion of Explained Variance [0.72962445 0.22850762 0.03668922 0.00517871]
```

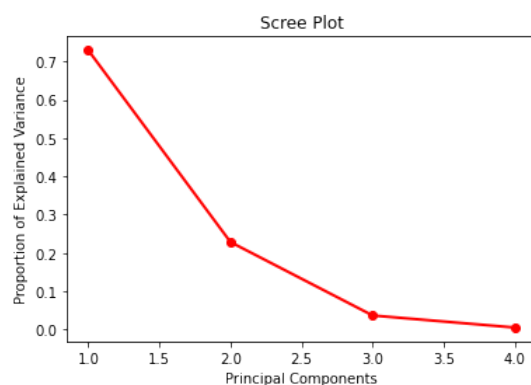
```
In [15]: import numpy as np
print('Cumulative Proportion of Explained Variance', np.cumsum(pca.explained_variance_ratio_)) # Cumulative Sum of t
x_pca['target']=y
x_pca.columns=['PC1', 'PC2', 'PC3', 'PC4', 'target']
x_pca.head() #printing with proper header values
#pc1 = 72 pc1+pc2=96 1+2+3=99
#pc1 and pc2 are important as they cover 96
```

Cumulative Proportion of Explained Variance [0.72962445 0.95813207 0.99482129 1.]

Out[15]:

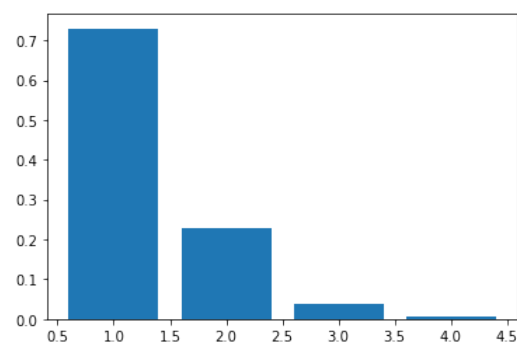
	PC1	PC2	PC3	PC4	target
0	-2.264703	0.480027	-0.127706	-0.024168	0
1	-2.080961	-0.674134	-0.234609	-0.103007	0
2	-2.364229	-0.341908	0.044201	-0.028377	0
3	-2.299384	-0.597395	0.091290	0.065956	0
4	-2.389842	0.646835	0.015738	0.035923	0

```
In [16]: import matplotlib.pyplot as plt
#it has 4 components therefore, 4-- 0 1 2 3
#for graph +1 -- 1 2 3 4
PC_values = np.arange(pca.n_components_)+1 #
#every component ka ratio
plt.plot(PC_values,pca.explained_variance_ratio_,'ro-',linewidth=2)
#r for red
#o for .
#- for line
plt.title("Scree Plot")
plt.xlabel("Principal Components")
plt.ylabel("Proportion of Explained Variance")
plt.show()
```



```
In [17]: plt.bar(PC_values,pca.explained_variance_ratio_) #bar plot
```

Out[17]: <BarContainer object of 4 artists>



```
In [18]: loadings=pd.DataFrame(pca.components_.T,columns=[f'PC{i+1}' for i in range(pca.n_components_)],index=x.columns)
loadings
```

Out[18]:

	PC1	PC2	PC3	PC4
0	0.521066	0.377418	-0.719566	-0.261286
1	-0.269347	0.923296	0.244382	0.123510
2	0.580413	0.024492	0.142126	0.801449
3	0.564857	0.066942	0.634273	-0.523597

